46-926, Fall 2020: Homework #6

Due **3:20pm**, December 16, 2019.

Homework should be submitted electronically on canvas before the deadline. Please submit your homework as a single file (pdf preferred, canvas struggles with html). We recommend using jupyter notebooks to prepare your homework.

Do all computational problems in python unless otherwise noted. Please post any questions on the Piazza discussion board.

1. *Exploring bias and variance for decision trees.* We're going to look at the bias and variance of a tree in one dimension, so that we can plot it. There are some starter functions in `problem1.py`. You'll find `get_data(n)`, which generates `n` data points, and `true_mean(x)`, which returns the *true* conditional mean at each point in x.

   When we think about the bias of linear regression, it's easy to imagine what we mean by $\mathbb{E}\widehat{\mu}(x)$ (the expected estimated fit) because the average of many linear fits is a linear fit. This is harder to visualize for trees because the average of many small trees is not a small tree! In this picture, we're going to simulate to plot the actual average of many trees as we vary depth, as well as the variance around that average.

   (a) Sample a data set of size 50 using `get_data`. Plot this data. Fit a tree with a single split in it. You're using regression trees rather than classification trees, so you want to use `sklearn.tree.DecisionTreeRegressor`. Otherwise it works just like our tree demo in class. You can set `max_depth = 1` in the call to `DecisionTreeRegressor` to get a single split tree. Overlay the true conditional mean on this same plot using the `true_mean` function. You should see that your tree is a pretty poor approximation to the true mean.

   > **Hint:** To plot all of the curves in this problem, define a grid of x points that spans $[0, 1]$, and evaluate each function or predictor on that grid. Then just plot the evaluations against the grid. This will let you visualize your tree fits and their averages (part b). You may find the function `np.linspace` useful here.

   (b) Now repeat the previous the previous simulation 100 times. For each simulation, you will draw a new data set and compute a new depth 1 tree. Plot the fits from

all of these trees together on the same plot (lots of lines). On top of this, plot the true conditional mean. Also plot the average of all of your trees as a single curve. Does the average of your trees look like the fit from a simple tree? Does it approximate the true conditional mean better than the step function you get from a single tree?

From these simulations, you can also compute the bias and variance directly. The bias is the difference between the average of your trees and the true mean function. The variance is the variance of your tree fits around their own mean. Compute both the bias and variance from your simulation and plot them as a function of x.

> **Hint:** You may find `np.mean` and `np.var` useful for summarizing your fits across the many trees. To use these, store all your predictions in a matrix of dimension (number of simulations) $\times$ (number of x grid points), and then use the above functions with the parameter `axis=0`.

(c) Now repeat parts (a) and (b), but allow your tree to have 5 splits.

i. Is the average of your trees a better approximation to the true mean function?

ii. Consider the spread of your individual trees around their average. How does it compare to the equivalent spread in part (b)?

2. *Random forests, marketing, and imbalanced data* Let's revisit the marketing data from last homework, this time with random forests. For comparisons in this problem, you'll want the scores for logistic regression from the previous homework.

Note: To use `sklearn`'s random forest function, you will need to transform your categorical variables to dummy coding first. Use the `get_dummies` in pandas. I recommend doing this before splitting your data, so you can do it once. I also recommend leaving all the variables in (the default), rather than dropping one to make it linearly independent.

(a) Let's see if you can do better than in previous homeworks. Fit a random forest to your training data using the `python RandomForestClassifier` function. Use 500 trees and set `oob_score=True`. Leave the rest of the parameters at their defaults for now.

i. What is your misclassification rate? Compare to the base rate and to the misclassification rate for your logistic classifier.

How does your misclassification rate compare to your OOB error for this forest?

ii. Plot an ROC curve overlaying all three classifiers and compare their performance. Include a legend. Also compare the AUC of your classifier to those of the other classifiers.

iii. How good is your top 1000 set for this classifier, and how does it compare to logistic regression?

(b) Let's try a different node size. This is one of the parameters that can actually have an impact on random forest fits, especially in imbalanced settings. Repeat the previous part, but set the `min_samples_leaf` parameter to 10.

(c) Let's see if we can improve further! The naive random forest can suffer when samples are highly imbalanced, since splits will tend not to favor the rare class, and since leaf nodes will usually be dominated by the common class. A variety of adjustments have been proposed to help correct for this. One of the simplest approaches is to shift the weight given to each class (similar to the weights we have discussed in linear regression, except only based on outcome class).

You can implement this in python by setting the `class_weight` parameter to `balanced`. This adds weight proportionally to the rarer class.

Fit this balanced version of the random forest. Again, compare misclassification, AUC, and top 1000 set. Create a new ROC plot showing all four classifiers. How do the two random forest classifiers compare?

Do you notice something strange about your misclassification rate here? Why do you think that is?

3. Now that we have a somewhat decent classifier, let's try to get a peek at how it works.

(a) Create a variable importance plot for the random forest model from problem 2b. What are the two most important variables to your classifier? (Note: In the class example I forgot to label the x-axis bins. You should add labels so the plot actually means something.)

(b) For the two most important variables, create univariate partial dependence plots. We'll discuss these next class, but for now you can think of them as a rough measure of how your predictions change "on average" as a function of one of your input variables. How do your predictions seem to depend on these two important variables, roughly?

Hints:
The random forest in sklearn does not have its own partial dependence plot function. However, the package `pdpbox` can be used to create a partial dependence plot for a given function. Basic usage is as follows:

```
from pdpbox import pdp
pdp_obj = pdp.pdp_isolate(model=your_model, dataset=data,
model_features=your_features, feature=target_feature)
fig, axes = pdp.pdp_plot(pdp_obj, 'Feature Name')
```

where `your_model` is the fitted model, `data` is a data set, `model_features` is a list of all the features you fed your classifier (can get this with `X.columns` for an X matrix), and `feature` is the feature you want to examine.

You can see more examples at `https://github.com/SauceCat/PDPbox/blob/master/tutorials/pdpbox_binary_classification.ipynb`

The package information and installation information can be found at `https://pdpbox.readthedocs.io/en/latest/`. You should be able to install it through your python package manager.

4. *Bad validation.* Here we demonstrate one way that validation of ML methods can go wrong. This ties back to some earlier discussions about cross-validation, and also leads into some practical discussions we will have in our last lecture.

   This problem will walk you through a common (but wrong) form of validation. *Don't do this in real life.*

   The file `bad_valid.py` contains starter code for this problem. You will use the function `get_data` to generate your data sets.

   (a) Generate a data set ($n = 100$) using the `get_data` function. We will be using this data set for both training and testing by using a validation split.

   First, you decide to investigate how useful your variables are for understanding y. Compute a vector of correlations between each column of X and the outcome vector y.

   You can compute the correlation between `y` and a single column of `X` using the `pearsonr` function from `scipy.stats`. I recommend using `np.apply_along_axis` to compute correlations with all the columns at once.

Plot a histogram of the correlations between y and each of the columns of `X` in your training set.

You should notice a nice bell-shaped curve of correlations around zero. (You might expect this in real life, since you know that you are just analyzing noise.)

(b) Suppose you decide that you have too many variables. Since you've noticed that most of the variables are worthless, you decide to discard those worthless variables before running your fancy ML methods.

Figure out which columns of `X` have absolute value correlation with `y` bigger than 0.2. Make a new data frame that only has those highly correlated columns; call it `X_good`. (You will probably want to keep track of the indices of these columns, since you'll need to make this matrix again later.)

(c) Split your `X_good` data set into the first `0.8*n` points (training set) and the last `0.2*n` points (test set). Think of this like back testing: you're going to train on the older data and predict on the more recent data.

Train a lasso model on the first data set, using the `LassoCV` function from `sklearn.linear_models`, with cross-validation to pick your regularization parameter.

What is the cross-validation estimate for your model's mean squared error? Use your model to predict on the test portion of the `X_good` data set. What is your test mean squared error?

At this point, everything should be looking pretty good!

Hints:
To get the cross-validated MSE, note that the `mse_path_` object contains all your cross-validated MSEs for each regularization parameter value and every fold separately. Taking averages across the rows (folds) will give you the CV-MSE at each regularization parameter value. The minimum of these is your cross-validated MSE for the selected model.

(d) Generate another data set of size 100 from `get_data` and assess your model on this fresh data. You will have to first extract the same columns that you extracted when you formed `X_good`, and then use your lasso model to predict with those subset columns. What is your mean squared error on this fresh data?

Explain why this number is quite different from your cross-validated error and your test error on the previously held-out data set?